Different Types of Flutter Widgets

Introduction to Multiplatform – Semester VIII

Dr Mohamad AOUDE

Lebanese University
Faculty of Engineering – Branch II

Different Types of Widgets

Output & Input (Visible)

`ElevatedButton()`, `Text()`, `Card()`, ...

Layout & Control (Invisible)

`Row()`, `Column()`, `ListView()`, ...

`Container()`

Drawn onto the screen: "What the user sees"

Give your app structure and control how visible widgets are drawn onto the screen (indirectly visible)

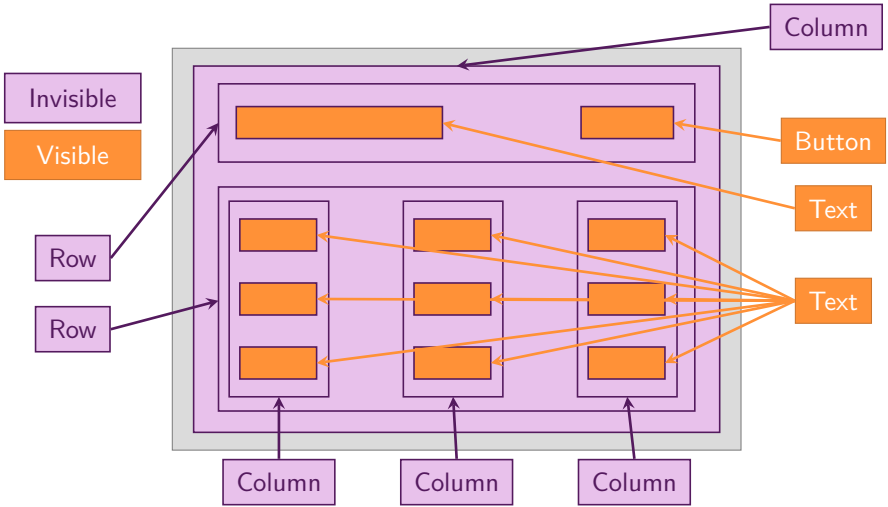
The source uses `RaisedButton()`; here the example is `ElevatedButton()`.

Details

Visible and Invisible Widgets

Everything in a Flutter UI is a widget, but widgets play two different roles. **Visible** (output & input) widgets such as `Text`, `ElevatedButton` or `Card` are drawn on the screen: they are what the user sees and interacts with. **Invisible** (layout & control) widgets such as `Row`, `Column` or `ListView` are not drawn themselves; they give the app its structure and decide where and how their visible children appear. `Container` sits in between: it is a layout widget, but it becomes visible as soon as you give it a color, border or decoration.

Visible vs Invisible Widgets

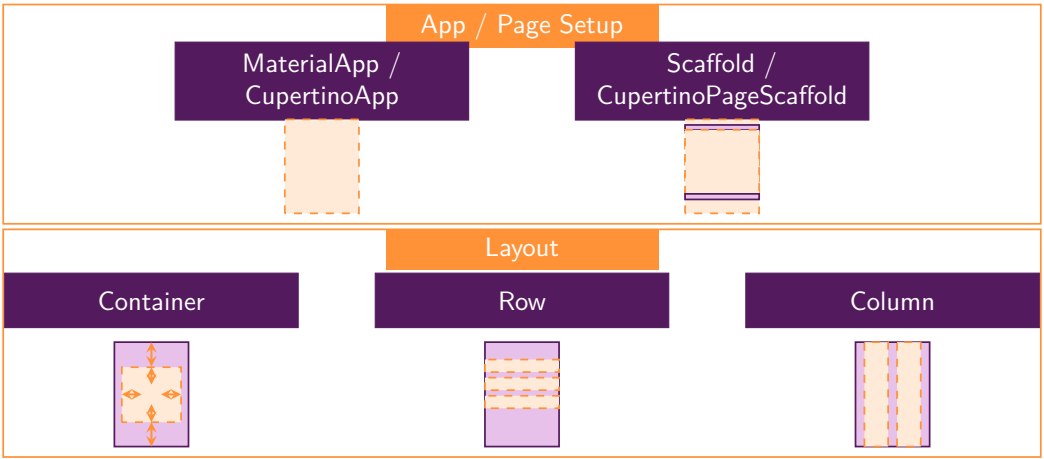


Details

Reading a Screen as Nested Widgets

Any screen can be read as a set of nested widgets. Here an outer `Column` stacks two `Rows` vertically. The first `Row` places a `Text` and a `Button` side by side; the second `Row` places three `Columns` side by side, each holding three `Text` widgets. Only the `Text` and `Button` widgets are actually drawn; the `Rows` and `Columns` are invisible and only control positions. Building a UI means choosing and nesting layout widgets until the visible widgets land where you want them.

Most Important Widgets



Details

MaterialApp / CupertinoApp

- Typically the root widget in your app.
- Performs much of the behind-the-scenes setup for your app.
- Allows you to configure a global theme for your app.
- Sets up navigation behavior, including page-transition animations.

Scaffold / CupertinoPageScaffold

- Typically used as the frame for a page in your app.
- Provides page structure: background, app bar or navigation bar; `Scaffold` can also provide bottom navigation and other controls.
- Normally use one scaffold per page.

Details

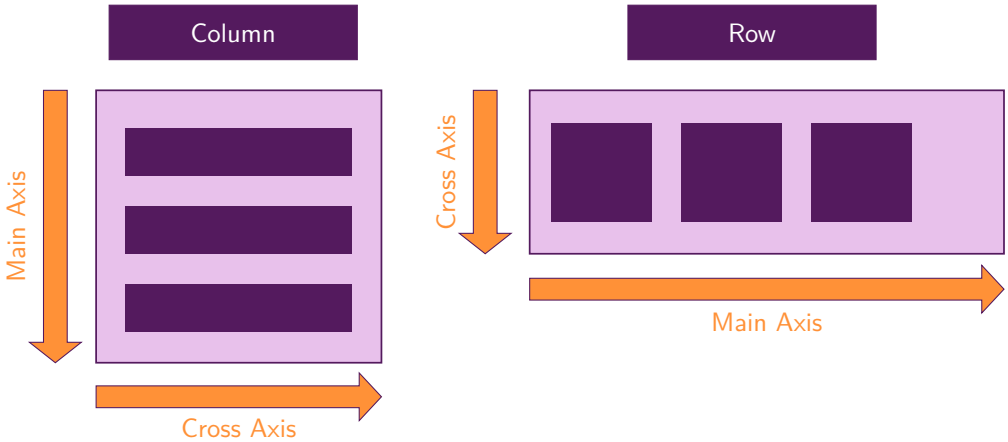
Container

- An extremely versatile widget.
- Can be sized with `width`, `height` or `constraints` (e.g. `BoxConstraints(maxWidth: ...)`); styled with `color` or a `decoration` (border, shape, gradient and more).
- Can take an optional child, which you can align in different ways.
- You will use this widget quite often.

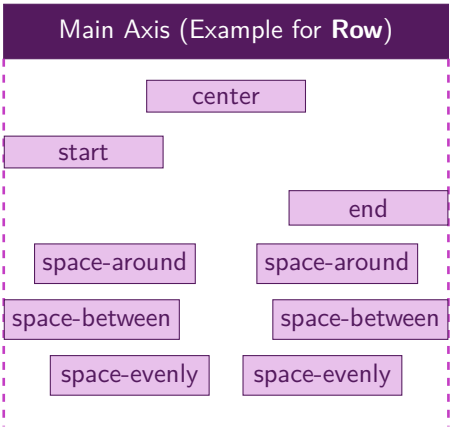
Row / Column

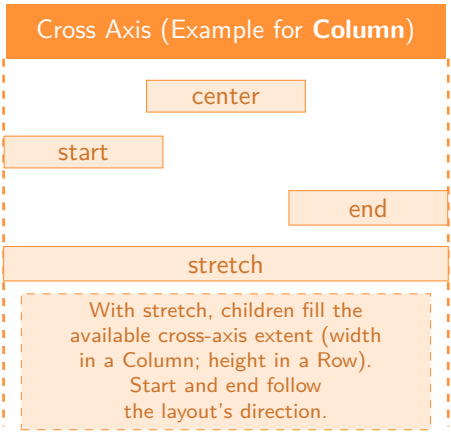
- Place multiple widgets next to each other horizontally (`Row`) or vertically (`Column`).
- Have limited styling options: wrap the layout or its children in a `Container` to apply styling.
- Align children along the main axis and the cross axis.

Column & Row



Alignments


Main Axis (Example for **Row**)
center
start
end
space-around
space-around
space-between
space-between
space-evenly
space-evenly

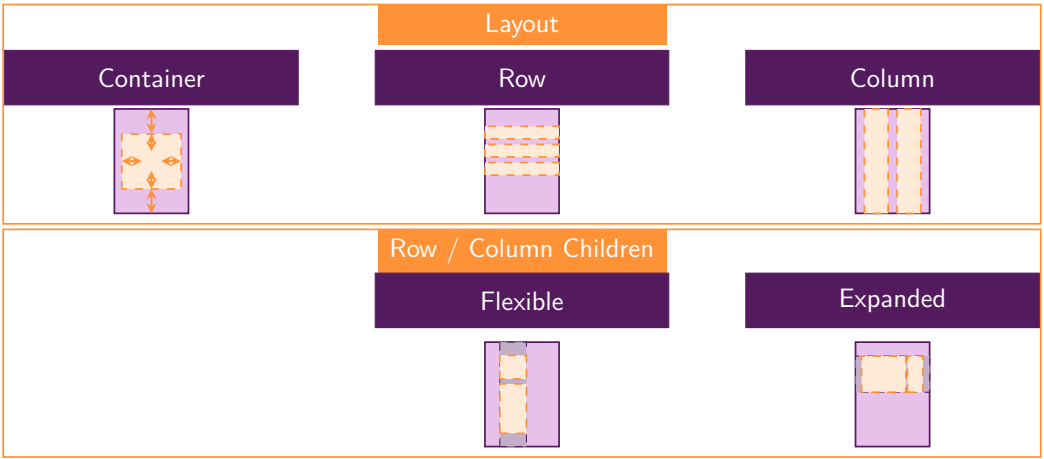

Cross Axis (Example for **Column**)
center
start
end
stretch
With stretch, children fill the available cross-axis extent (width in a Column; height in a Row). Start and end follow the layout's direction.

Details

Axes and Alignment

Every `Row` and `Column` has two axes. The **main axis** is the direction in which the children are placed: horizontal for a `Row`, vertical for a `Column`. The **cross axis** is perpendicular to it. `mainAxisAlignment` distributes the children along the main axis (`start`, `end`, `center`, `spaceBetween`, `spaceAround`, `spaceEvenly`), while `crossAxisAlignment` positions them on the cross axis (`start`, `end`, `center`, `stretch`, and `baseline` for text in a `Row`). `mainAxisSize` decides whether the layout takes all available space or only what its children need.

Most Important Widgets



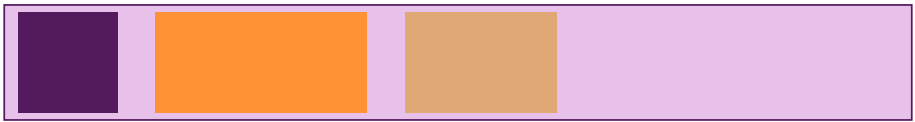
Details

Flexible / Expanded

- Help you size the child widgets of a `Row` or `Column`.
- `flex` sets each child's share of the remaining space (default 1).
- `Flexible` lets the child be smaller than its share (`FlexFit.loose`); `Expanded` is `Flexible` with `FlexFit.tight`: the child must fill it.

Flexible & Expanded

Row with Flexible(fit: **FlexFit.loose**) on all children



Children may use less than their allocated share, based on their content or size.

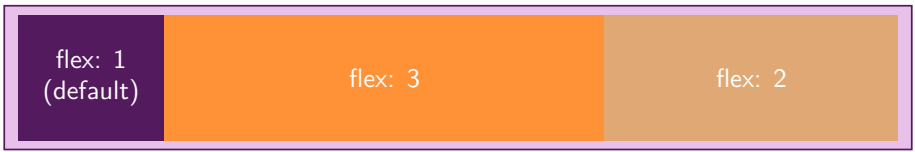
Row with Flexible(fit: **FlexFit.tight**) or **Expanded** on all children



With equal flex values, children fill equal shares of the available space.

Flexible & Expanded

Row with Flexible(fit: **FlexFit.tight**) or **Expanded** on all children

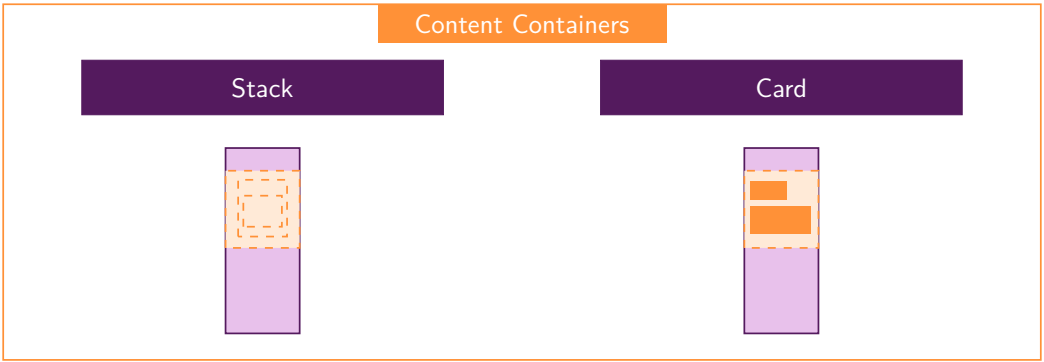


Children fill the available space in proportion to their flex values.

Add all flex values; each child receives its fraction of the total.

Example: the orange child receives $3/(1 + 3 + 2) = 3/6$ of the space.

Most Important Widgets



Details

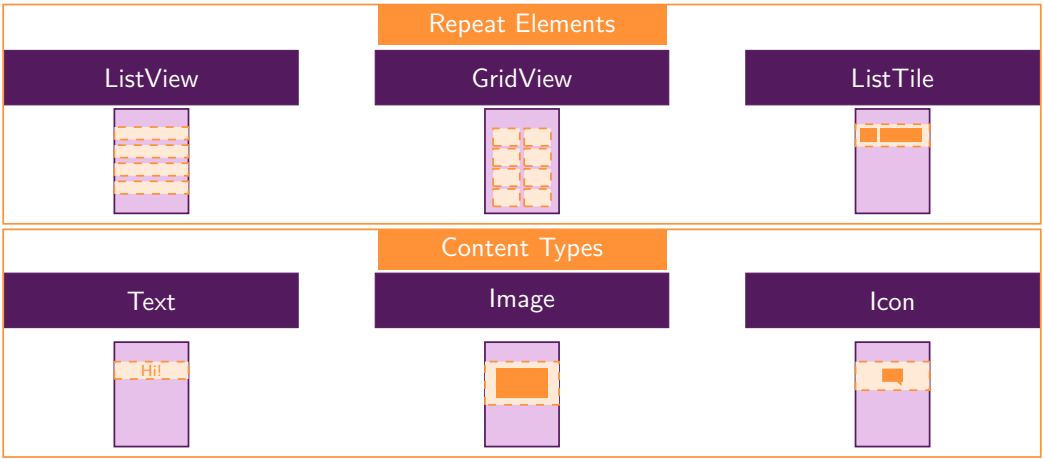
Stack

- Used to position items on top of each other (along the Z axis).
- Widgets can overlap.
- Use `Positioned()` to place children at offsets within the stack.

Card

- A container with default styling: shadow, background color and rounded corners.
- Can take one child, which can be any widget.
- Typically presents a single piece or group of information.

Most Important Widgets



Details

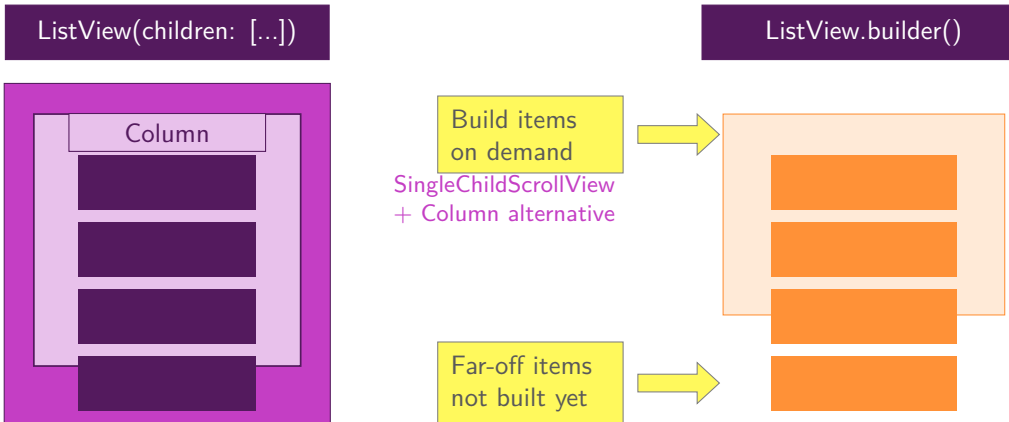
ListView / GridView

- Display lists or grids of items.
- Provide scrolling; a `Column` does not scroll by itself.
- Can scroll vertically (the default) or horizontally.
- Use `ListView.builder()` for on-demand item creation in very long lists.

ListTile

- A pre-styled widget that provides a typical list-item appearance.
- Offers slots for widgets: leading, title, subtitle and trailing.
- Optional, but useful for a standard list-item layout.

ListView



The children constructor creates the widget list eagerly. The builder creates items as needed, including a cache near the viewport.

Details

Text

- Outputs text on the screen.
- Text can be styled: font family, font weight, font size and more.
- Overflow behavior can be controlled, such as clipping text that is too long.

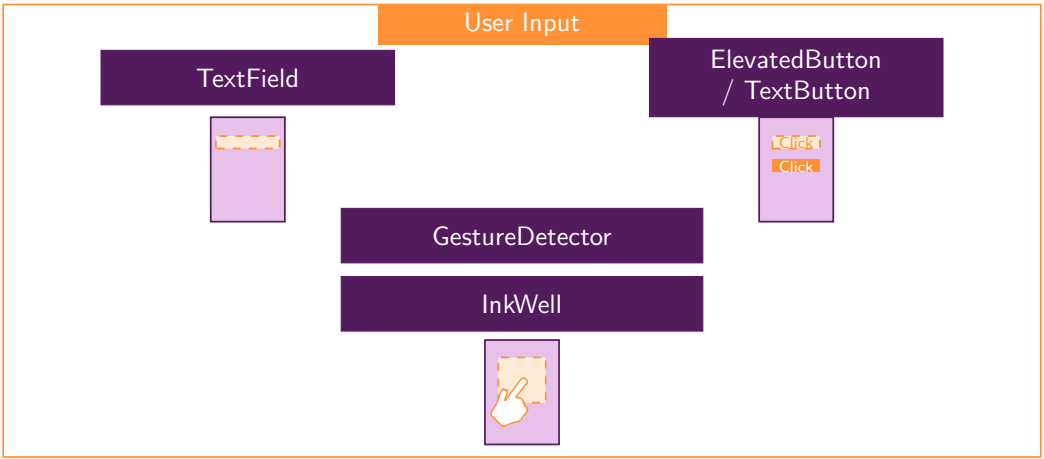
Image

- Renders an image on the screen.
- Supports different sources, including app assets and web images.
- Can fit an image into its containing area in different ways.

Icon

- Renders an icon on the screen.
- Material icons are built in; Cupertino icons need the `cupertino_icons` package.
- Use `IconButton()` when you need a button with an icon.

Most Important Widgets



Source names: `RaisedButton` / `FlatButton`; this deck uses `ElevatedButton` / `TextButton`.

Details

TextField

- Provides an editable text field for users to enter information.
- Offers many options: autocorrection, error messages, labels and styles.
- Supports keyboard types such as email, number and normal text.

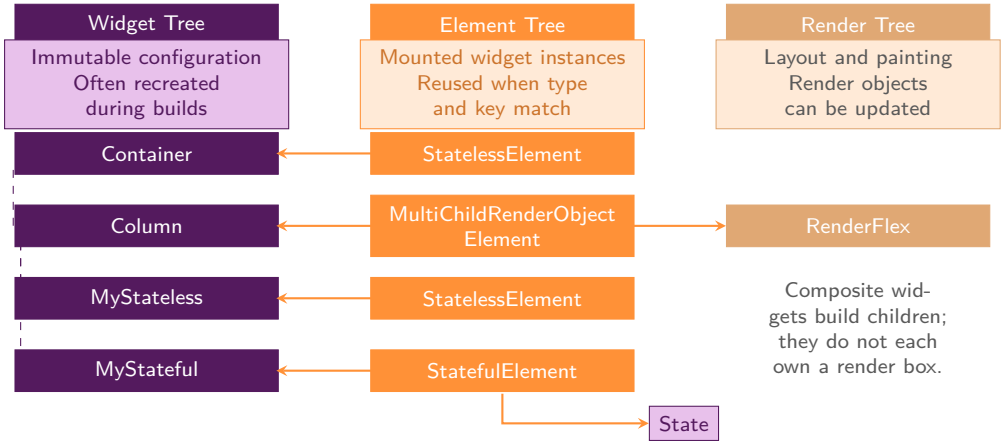
ElevatedButton / TextButton / IconButton

- Differently styled buttons that handle user taps.
- Provide an `onPressed` callback for the action; a null callback disables the button.
- Buttons can be styled and customized.

GestureDetector / InkWell

- `GestureDetector` wraps a widget with gesture listeners, such as double tap or long press.
- `InkWell` supports taps and visual ink effects on a Material surface.
- Use these widgets to create custom buttons or touchable content.

Widget Tree & Element Tree



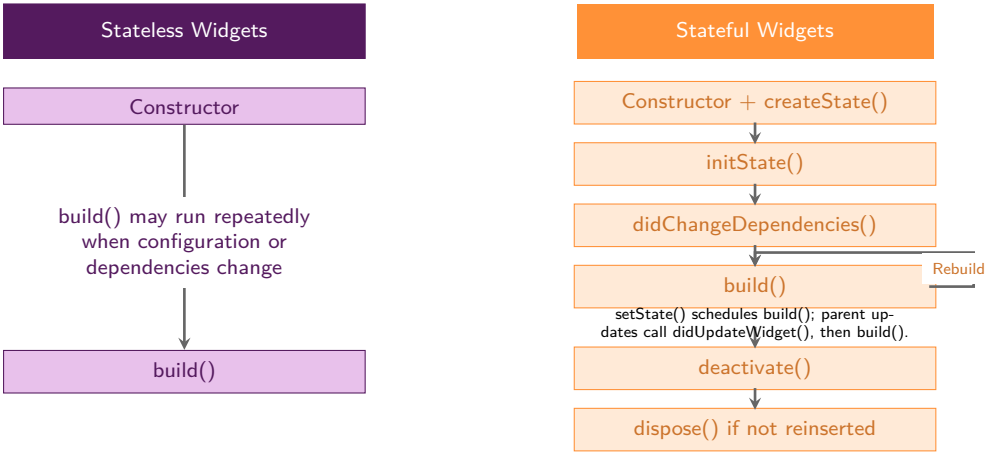
Schematic: intermediate widgets omitted. Flutter architectural overview

Details

Three Trees

Flutter keeps three parallel trees. The **widget tree** is what your `build()` methods return: lightweight, immutable descriptions that are recreated often. The **element tree** holds the mounted instances; on a rebuild, Flutter compares each new widget with the old one and reuses the element when the type and key match. For a `StatefulWidget`, the element also keeps the `State` object alive across rebuilds. The **render tree** holds the render objects (e.g. `RenderFlex` for a `Row` or `Column`) that do layout and painting. This is why rebuilding widgets is cheap: only what actually changed is updated on screen.

Widget Lifecycle



Simplified State lifecycle. Flutter API: State

Stateless vs Stateful Lifecycle

A `StatelessWidget` only has a constructor and `build()`; `build()` runs again whenever the parent rebuilds it with new configuration or an inherited dependency changes. A `StatefulWidget` creates a separate `State` object through `createState()`. `initState()` runs once for setup (controllers, subscriptions), `didChangeDependencies()` runs when inherited data such as the theme changes, and `build()` produces the UI. Calling `setState()` marks the state as dirty so `build()` runs again. When the widget is removed, `deactivate()` is called, and `dispose()` releases resources if it is not reinserted.

App Lifecycle

Lifecycle State Name	When is it hit?
`resumed`	Visible and has input focus
`inactive`	Visible, but does not have input focus
`hidden`	All app views are hidden
`paused`	Not visible; frame call-backs stop (Android / iOS)
`detached`	Engine running without an attached host view

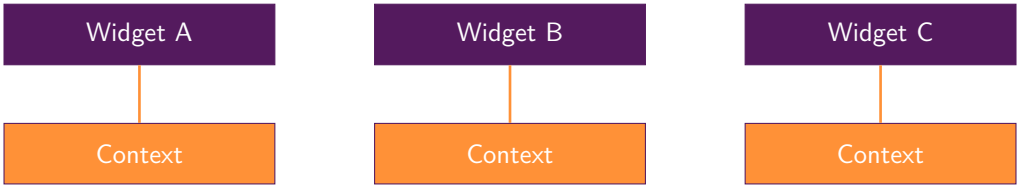
Source update: `suspending` is an obsolete name. Lifecycle notifications are not guaranteed before termination.

Flutter API: AppLifecycleState

Reacting to the App Lifecycle

The app lifecycle describes the whole application, not a single widget. The operating system moves the app between states: `resumed` when it is in the foreground and receives input, `inactive` when it is visible but not focused (e.g. during a phone call or in the app switcher), `hidden` and `paused` when it goes to the background, and `detached` when the engine runs without a view. You can listen to these changes with `AppLifecycleListener` or `WidgetsBindingObserver` to pause animations, save data or release resources, but you should not rely on a notification arriving before the app is terminated.

Understanding “Context”



Information about a widget’s location in the widget tree

Material Theme

Navigator

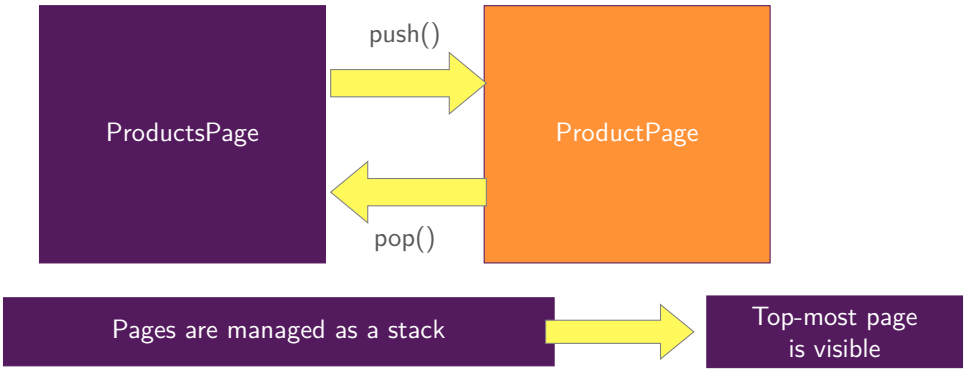
`Theme.of(context)` and `Navigator.of(context)` look up ancestors from that location.

Details

What BuildContext Gives You

Every widget receives a `BuildContext` in its `build()` method. The context is the widget's handle to its own position in the widget tree, so through it the widget can reach all of its ancestors. Methods such as `Theme.of(context)`, `MediaQuery.of(context)` or `Navigator.of(context)` use it to walk up the tree and find the nearest ancestor of a given type. This is why the same call can return different results in different parts of the app, and why the context must belong to a widget located *below* the ancestor you are looking for.

Navigation in Flutter Apps

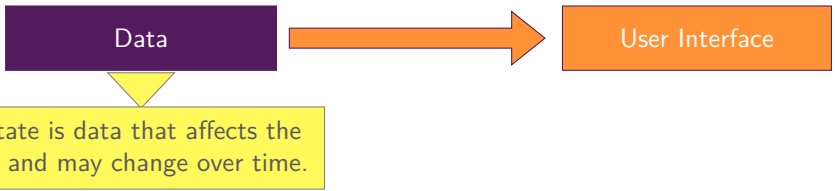


The Navigator Stack

A Flutter app manages its screens (routes) with a `Navigator` that works like a stack. `Navigator.push()` places a new page on top of the stack, and only the top-most page is visible. `Navigator.pop()` removes the top page and returns to the one below it; it can also send a result back to the previous page. Pages are usually pushed with a `MaterialPageRoute` or by name with `pushNamed()`, and the back button of the `AppBar` or of the system calls `pop()` automatically.

What's "State" and "State Management"?

The user interface is a **function of your data (state)**

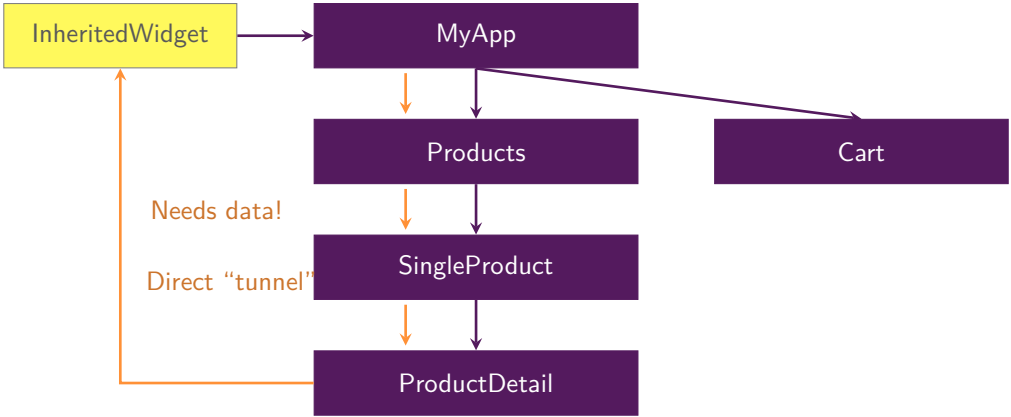


App-wide State	Widget (Local) State
Affects the app or significant parts of it	Needed only within a widget or its subtree
Authentication, shared products, cart contents, ...	A local loading spinner, form input, ...

Local vs App-wide State

In Flutter the UI is rebuilt from the current data, so changing the data is how you change the screen. Data that affects the UI and can change over time is called **state**. **Widget (local) state** belongs to one widget and its subtree, such as the text of a form field or a loading flag, and is usually handled with a `StatefulWidget` and `setState()`. **App-wide state** is shared by many screens, such as the logged-in user or the cart contents. Passing it manually through constructors quickly becomes hard to maintain, which is why state management solutions are used.

The InheritedWidget



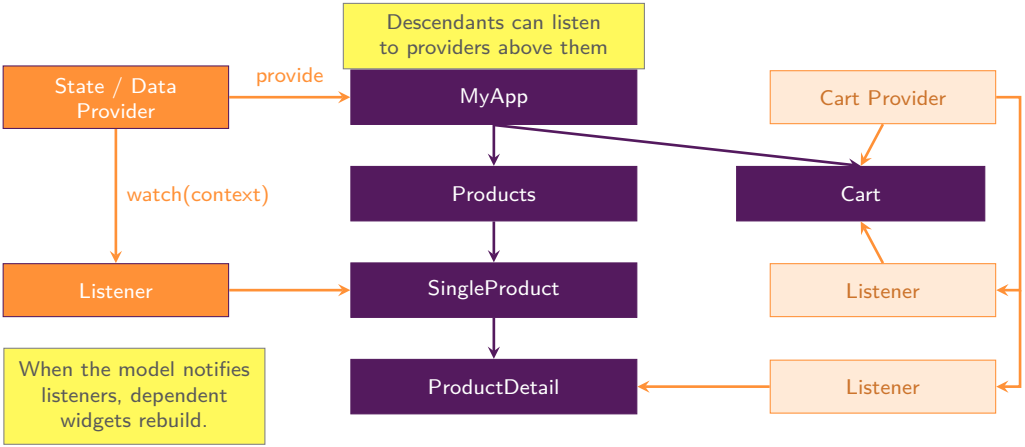
A descendant can depend on an inherited ancestor without passing data through every constructor.

Details

Sharing Data with InheritedWidget

Without help, data needed deep in the tree must be passed through the constructors of every intermediate widget, even those that never use it. An `InheritedWidget` placed high in the tree solves this: any descendant can reach it directly with `context.dependOnInheritedWidgetOfExactType()`, usually wrapped in a static `of()` method, as in `Theme.of(context)`. The descendant is also registered as a dependent, so it is rebuilt automatically when the `InheritedWidget` is replaced with new data.

The Provider Package & Pattern



Schematic: providers must wrap the descendants that read them.

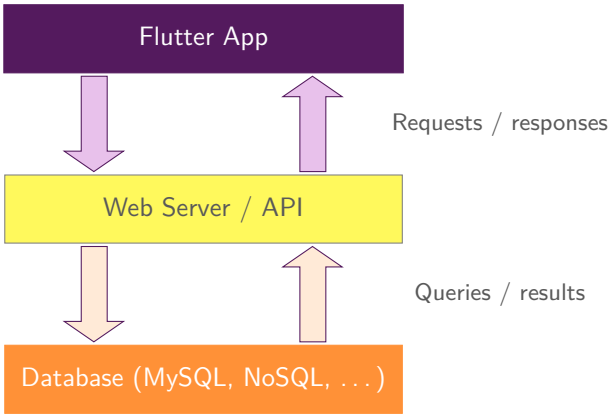
Details

State Management with Provider

The `provider` package builds on `InheritedWidget` and makes it easier to use. A data class, usually extending `ChangeNotifier`, holds the state and calls `notifyListeners()` when it changes. A `ChangeNotifierProvider` placed above the widgets that need the data makes it available to them. Descendants use `context.watch<T>()` to read the data and rebuild on changes, or `context.read<T>()` to call methods without listening. Only listening widgets are rebuilt, and several providers can be combined with `MultiProvider`.

Connecting Flutter to a Database

Direct access with privileged database credentials exposes them in the app.



The API authenticates users and controls access. Managed database SDKs can use enforced access rules.

Why an API in Between?

A Flutter app should not connect directly to a database such as MySQL: the database credentials would have to be shipped inside the app, where anyone can extract them. Instead, the app sends requests to a web server (API) that authenticates the user, checks permissions, runs the database queries and returns the results, usually as JSON. Managed backends such as Firebase follow the same idea: their SDKs talk to the service, which enforces security rules on the server side.

HTTP Requests

When communicating with a (REST) API

Convention: **HTTP endpoint (URL) + HTTP method = action**

GET	POST	PATCH	DELETE
Fetch data	Submit / create data	Partially update data	Delete data
		PUT	
		Replace / create at URL	

HTTP Methods in Practice

A REST API identifies resources by URLs (endpoints) and uses the HTTP method to say what to do with them: `GET` reads data, `POST` creates new data, `PUT` replaces a resource, `PATCH` changes part of it and `DELETE` removes it. For example, `GET /products` returns all products and `DELETE /products/5` removes product 5. In Flutter these requests are commonly sent with the `http` package, which returns a response with a status code and a body that is usually decoded from JSON.